메타월드 웹 기반 멀티플레이 프로토타입 기술계획서

1. 프로젝트 개요

1.1 프로젝트명

Meta World (메타월드)

1.2 프로젝트 목적

웹 브라우저에서 실행 가능한 3D 메타월드 프로토타입을 개발한다.

사용자는 별도의 프로그램 설치 없이 웹사이트에 접속하여 가상의 자연 월드에서 캐릭터를 조작하고, 다른 사용자와 실시간으로 만나 이동하며 채팅할 수 있다.

본 프로젝트는 복잡한 게임 시스템을 제외한 최소 기능의 프로토타입을 우선 개발하고, 향후 다양한 소셜 기능 및 게임 기능을 확장할 수 있도록 설계한다.

1.3 개발 방향

- Low Poly 3D 기반의 가벼운 그래픽 구현
- 웹 브라우저 기반 게임 실행
- 3인칭 캐릭터 조작
- 실시간 멀티플레이 환경 구현
- 실시간 채팅 기능 구현
- 로컬 개발 환경에서 제작 후 GitHub 및 Vercel을 통해 공개
- 복잡한 게임 기능을 배제한 단계적 개발

---

2. 확정 기술 사양

항목	확정 사양
그래픽 스타일	Low Poly 3D
카메라	Third Person
최초 월드	Nature
개발 언어	JavaScript / TypeScript

런타임	Node.js
웹 프레임워크	Next.js
3D 엔진	Three.js
3D React 연동	React Three Fiber
3D 보조 라이브러리	Drei
실시간 통신	Socket.IO
게임 서버	Node.js
서버 프레임워크	Express
웹사이트 배포	Vercel
실시간 서버 배포	Render 또는 Railway
코드 저장소	GitHub

---

3. 프로토타입 구현 범위

3.1 기본 월드

최초 월드는 Nature 월드로 구성한다.

주요 구성 요소:

- Low Poly 지형
- 잔디 및 나무
- 바위 및 자연 오브젝트
- 하늘 및 환경 조명
- 플레이어가 자유롭게 이동할 수 있는 공간

3.2 플레이어 시스템

- 기본 캐릭터 생성
- 3인칭 카메라
- WASD 이동
- 마우스 카메라 회전
- 점프
- 캐릭터 이동 및 방향 전환
- 플레이어 닉네임 표시

3.3 실시간 멀티플레이

- 여러 사용자의 동시 접속
- 플레이어별 고유 ID 부여
- 다른 플레이어 캐릭터 실시간 표시
- 플레이어 이동 동기화
- 플레이어 접속 및 퇴장 처리
- 동일 월드 내 플레이어 목록 관리

3.4 실시간 채팅

- 게임 화면 내 채팅 UI
- 닉네임 및 메시지 표시
- 메시지 전송 및 수신
- Enter 키를 통한 메시지 전송
- 다른 접속자에게 실시간 메시지 전달

3.5 프로토타입 제외 기능

개발 범위를 단순화하기 위해 다음 기능은 초기 버전에서 제외한다.

- 회원가입 및 로그인
- 데이터베이스 기반 사용자 정보 저장
- 아이템 및 인벤토리
- 게임 내 경제 시스템
- 월드 제작 및 편집
- 친구 및 그룹 시스템
- 월드 이동 및 포털
- 음성 채팅
- 복잡한 게임 플레이 및 퀘스트

---

4. 시스템 아키텍처

4.1 전체 시스템 구성

시스템은 웹 클라이언트와 실시간 게임 서버를 분리하여 구성한다.

웹 클라이언트

- Next.js
- Three.js
- React Three Fiber
- Drei

실시간 게임 서버

- Node.js
- TypeScript
- Express
- Socket.IO

배포 환경

- Vercel: 웹 클라이언트
- Render 또는 Railway: 실시간 게임 서버
- GitHub: 소스 코드 관리

4.2 시스템 동작 구조

1. 사용자가 웹사이트에 접속한다.
2. 닉네임을 입력하고 Nature 월드에 입장한다.
3. 브라우저가 Socket.IO 서버에 연결한다.
4. 서버가 플레이어 ID를 부여하고 접속 정보를 관리한다.
5. 접속한 플레이어의 캐릭터가 월드에 표시된다.
6. 플레이어가 이동하면 위치 정보가 서버로 전달된다.
7. 서버는 해당 월드의 다른 플레이어에게 이동 정보를 전달한다.
8. 다른 플레이어의 브라우저에서 캐릭터 위치가 갱신된다.
9. 채팅 메시지도 동일한 실시간 통신 서버를 통해 전달된다.

4.3 플레이어 이동 데이터

이동 동기화에 사용하는 데이터는 최소화한다.

예시:

```
None
{
  playerId: "player-001",
  nickname: "Player01",
  position: {
    x: 10.5,
    y: 0,
    z: 4.2
  },
  rotation: 1.57
}
```

캐릭터 모델 및 3D 월드 전체를 전송하지 않고, 플레이어의 위치와 회전값만 전달한다.

이를 통해 실시간 통신량을 줄이고 브라우저에서 부드러운 이동을 구현한다.

---

5. 서버 개발 계획

5.1 서버 역할

Socket.IO 서버는 다음 기능을 담당한다.

- 사용자 접속 및 연결 관리
- 플레이어 ID 및 닉네임 관리
- 월드 접속자 목록 관리
- 플레이어 이동 정보 수신
- 플레이어 이동 정보 전달
- 접속 종료 및 퇴장 처리
- 실시간 채팅 메시지 전달

5.2 서버 구성

```
None
server/
├── src/
│   ├── index.ts
│   ├── players.ts
│   ├── movement.ts
│   └── chat.ts
│
└── package.json
```

파일별 역할:

파일	기능
index.ts	서버 실행 및 Socket.IO 연결
players.ts	플레이어 접속 및 퇴장 관리
movement.ts	플레이어 이동 정보 처리
chat.ts	채팅 메시지 처리

5.3 실시간 통신 이벤트

이벤트	기능
player	플레이어 접속

player	접속 플레이어 목록
player	플레이어 이동 정보
player	플레이어 퇴장
chat	채팅 메시지 전달

초기 버전에서는 하나의 Nature 월드만 운영한다.

---

6. 프로젝트 폴더 구조

```
None
meta-world/
│
├── client/
│   ├── src/
│   │   ├── app/
│   │   ├── components/
│   │   └── game/
│   │
│   └── package.json
│
├── server/
│   ├── src/
│   │   ├── index.ts
│   │   ├── players.ts
│   │   ├── movement.ts
│   │   └── chat.ts
│   │
│   └── package.json
│
├── README.md
└── .gitignore
```

클라이언트 구성

폴더	역할

app	Next.js 웹페이지
components	게임 UI 및 화면 구성
game	3D 월드, 캐릭터 및 게임 로직

서버 구성

폴더	역할
players	플레이어 상태 관리
movement	이동 동기화
chat	채팅 처리
index	서버 실행

---

7. 개발 환경 및 배포 계획

7.1 로컬 개발 환경

개발 PC에 다음 프로그램을 설치한다.

- Visual Studio Code
- Node.js LTS
- Git
- GitHub 계정

로컬 개발은 Next.js 클라이언트와 Socket.IO 서버를 각각 실행하는 방식으로 진행한다.

예시:

```
None
웹 클라이언트
http://localhost:3000

게임 서버
http://localhost:3001
```

7.2 GitHub 저장소 관리

GitHub에 프로젝트 저장소를 생성하고 개발 단계별 변경사항을 업로드한다.

```
None
git add .
git commit -m "Update Meta World"
git push
```

GitHub는 프로젝트의 소스 코드 저장 및 버전 관리에 사용한다.

7.3 Vercel 배포

Next.js 클라이언트를 Vercel에 배포한다.

배포 과정:

1. Vercel 로그인
2. GitHub 저장소 연결
3. Next.js 프로젝트 선택
4. 환경변수 설정
5. Deploy 실행

7.4 실시간 게임 서버 배포

Socket.IO 서버는 Render 또는 Railway에 배포한다.

배포 후 웹 클라이언트의 서버 연결 주소를 공개된 게임 서버 주소로 변경한다.

예시:

```
None
웹사이트:
https://meta-world.vercel.app

실시간 서버:
https://meta-world-server.example
```

위 주소는 구성 예시이며 실제 배포 시 생성된 주소를 사용한다.

---

8. 단계별 개발 일정 및 작업 계획

STEP 1. 개발 환경 구축

- Node.js 설치

- VS Code 설치
- Next.js 프로젝트 생성
- Three.js 및 React Three Fiber 설치
- 로컬 실행 확인

STEP 2. 3D 자연 월드 구현

- Low Poly 지형 구성
- 나무 및 자연 오브젝트 배치
- 하늘 및 조명 설정
- 카메라 설정
- 기본 월드 렌더링

STEP 3. 플레이어 구현

- 기본 캐릭터 생성
- 3인칭 카메라 구성
- WASD 이동
- 마우스 카메라 조작
- 점프 기능
- 이동 및 충돌 처리

STEP 4. Socket.IO 서버 구현

- Node.js 서버 생성
- Socket.IO 연결
- 플레이어 접속 및 퇴장
- 플레이어 ID 및 닉네임 관리

STEP 5. 멀티플레이 구현

- 플레이어 목록 동기화
- 다른 플레이어 캐릭터 표시
- 이동 정보 실시간 전달
- 캐릭터 위치 및 회전 동기화
- 접속 종료 처리

STEP 6. 실시간 채팅 구현

- 채팅 UI 생성
- 메시지 입력 및 전송
- 메시지 서버 전달
- 다른 플레이어 메시지 수신
- 닉네임 및 메시지 표시

STEP 7. 로컬 테스트

- 브라우저 2개 이상 접속

- 플레이어 이동 테스트
- 캐릭터 동기화 테스트
- 채팅 테스트
- 접속 종료 테스트

STEP 8. GitHub 업로드

- 프로젝트 저장소 생성
- 소스 코드 업로드
- 버전 관리

STEP 9. Vercel 및 서버 배포

- 웹 클라이언트 Vercel 배포
- Socket.IO 서버 배포
- 서버 연결 설정
- 공개 웹사이트 테스트

---

9. 테스트 및 완료 기준

프로토타입은 다음 기준을 충족하면 1차 개발 완료로 판단한다.

테스트 항목	완료 기준
웹 실행	브라우저에서 정상 실행
3D 월드	Nature 월드 정상 표시
캐릭터 조작	WASD 및 카메라 조작 가능
멀티플레이 접속	2명 이상 동시 접속
캐릭터 표시	서로의 캐릭터 확인 가능
이동 동기화	이동이 다른 사용자에게 반영
채팅	메시지 실시간 전달
퇴장 처리	접속 종료 시 캐릭터 제거
배포	공개 웹사이트 접속 가능

---

10. 확장 개발 방향

프로토타입 완료 후 다음 기능을 단계적으로 확장할 수 있다.

- 사용자 회원가입 및 로그인
- 캐릭터 커스터마이징
- 월드 추가
- 친구 및 그룹 기능
- 음성 채팅
- 사용자 정보 및 월드 데이터 저장
- 월드 내 아이템 및 상호작용
- 월드 제작 및 편집 기능

초기 개발에서는 이러한 기능을 구현하지 않고, 실시간 멀티플레이와 채팅이 안정적으로 동작하는 것을 우선한다.

---

11. 최종 개발 목표

본 프로젝트의 최종 1차 목표는 다음과 같다.

웹 브라우저에서 실행되는 **Low Poly 3D** 자연 월드에 여러 사용자가 접속하여**, 3**인칭 캐릭터를 조작하고 서로의 캐릭터를 실시간으로 확인하며 채팅할 수 있는 메타월드 프로토타입을 제작한다**.**

개발은 로컬 PC에서 시작하여 GitHub에 소스 코드를 관리하고, Vercel을 통해 웹사이트를 공개한다. 실시간 게임 서버는 별도로 배포하여 멀티플레이 기능을 지원한다.

복잡한 게임 기능을 제외하고 핵심적인 소셜 메타월드 경험을 먼저 완성한 후, 향후 확장 가능한 구조로 개발한다.